CISC 322/326 Assignment 1

A Report on the Conceptual Architecture of WezTerm

October 9, 2026

Group: TheZucks

Authors:

Avaneesh Nitya-Nootan
23cjq3@queensu.ca

Arnav Dwivedi
23wvf4@queensu.ca

Dagim Weldu
23bbt14@queensu.ca

Ibraheem Syed
24skb@queensu.ca

Rahul Kumar
21rk74@queensu.ca

Armaan Chowdhury
23bl86@queensu.ca

Abstract

This report presents the conceptual architecture of WezTerm, an open-source, GPU-accelerated terminal emulator and multiplexer written in Rust. We derived the architecture by studying WezTerm's documentation and source repository alongside the course material on architectural styles, tracing two use cases through sequence diagrams, and combining the components and interactions those diagrams revealed into a single box-and-arrow diagram. The result has eight top-level subsystems: Entry/CLI/Bootstrap, Configuration & Lua, GUI & GPU Rendering, Font & Text Shaping, the Multiplexer (Mux), the Terminal Emulation Core, PTY & Process, and the Client–Server Layer.

The Mux sits at the centre of the system. It owns every session in a hierarchy of domains, windows, tabs and panes and exposes them through a common Pane interface, so the GUI draws local and remote sessions the same way. We identify two main architectural styles: client–server, which lets a headless mux server keep sessions alive after the GUI closes or the network drops, and implicit invocation (publish–subscribe), which lets the Mux notify the GUI of new output instead of having the GUI poll every pane.

Two use cases show the subsystems cooperating: typing a command in a local tab and seeing its output, and reconnecting to a persistent multiplexer session. We also evaluate two discarded alternatives, GUI-owned sessions and direct calls or polling, and close with lessons learned and a report on how our group worked with an AI teammate.

Contents

1 Introduction 3

2 Derivation Process 3

3 Conceptual Architecture 3

3.1 Architectural Tiers....4

3.2 Dependencies....4

3.3 Relation to the Reference Architecture....5

4 Subsystems and Responsibilities 5

4.1 Terminal Emulation Core....5

4.2 Font & Text Shaping....6

4.3 GUI & GPU Rendering....6

4.4 Multiplexer (Mux)....6

4.5 Client–Server Layer....7

4.6 PTY & Process....7

4.7 Configuration & Lua Engine....7

4.8 Entry / CLI / Bootstrap....8

5 Use Cases 8

5.1 Use Case 1: Typing a Command in a Local Tab....8

5.2 Use Case 2: Reconnecting to a Saved Multiplexer Session....9

6 Architectural Styles 10

6.1 Client–Server....10

6.2 Implicit Invocation (Publish–Subscribe)....11

6.3 Why Not Layered....11

7 Alternative Architectural Decisions 11

7.1 GUI-Owned Sessions Instead of Client–Server....11

7.2 Direct Calls or Polling Instead of Publish–Subscribe....11

8 Conclusion 11

9 Lessons Learned 12

10 Data Dictionary 12

11 Naming Conventions 12

References 13

Appendix D: AI Collaboration Report 14

D.1 Model Selection....14

D.2 Task Assignment....14

D.3 Interaction Protocol and Prompting Strategy....14

D.4 Validation and Quality Control....14

D.5 Quantitative Contribution....15

D.6 Reflection on Human–AI Collaboration....15

1 Introduction

A terminal emulator sits between a user and a command-line shell: it turns keystrokes into the bytes a shell expects, and turns the shell's output, a stream of text and escape sequences, into a picture on the screen. WezTerm, created by Wez Furlong, is a cross-platform terminal emulator for Linux, macOS and Windows. Besides the usual terminal features, it renders text on the GPU, is configured entirely through a Lua script, and includes a built-in multiplexer that splits windows into panes and can keep sessions running in a separate server process [1], [2].

These features make WezTerm a useful subject for architectural study. A single keystroke passes through the windowing system, the GUI, the multiplexer, the terminal model, a pseudo-terminal and a child shell, and the output returns along a different, asynchronous path. The multiplexer also lets a session be displayed by a GUI on another machine, which separates where a session lives from where it is shown.

Section 2 explains how we derived the architecture, Sections 3 and 4 present the conceptual architecture and its subsystems, and Section 5 walks through two use cases. Sections 6 and 7 discuss architectural styles and alternatives, and Appendix D reports how we worked with an AI teammate.

2 Derivation Process

We derived WezTerm's conceptual architecture in three steps, each building on the one before.

Step 1: Reading the documentation and source. We first studied what a terminal emulator does in general, using the course material on architectural styles [3] and WezTerm's own explanation of terminals, PTYs and shells [4]. We then read the official documentation on multiplexing, configuration and the Lua API, and examined the crate layout of the WezTerm GitHub repository [1]. Because the repository is already split into crates such as term, mux, wezterm-gui, wezterm-font, pty, config, codec and wezterm-client, we could group related crates into eight candidate subsystems. Each group member researched one or two of them (Section 4).

Step 2: Tracing use cases with sequence diagrams. A list of components does not show how they depend on each other, so we traced two use cases that exercise the system in different ways: typing a command in a local tab (Section 5.1) and reconnecting to a persistent session with wezterm connect (Section 5.2). For each one we used the Step 1 subsystems as lifelines and followed the documentation and source to find who calls whom, and in what order. The diagrams revealed behaviour the documentation alone did not. In Use Case 1, output does not reach the screen through one chain of calls: a reader thread in the Mux reads the shell's output, parses it into actions and applies them to the terminal model (steps 7–10), the Mux then announces that the pane changed (steps 11–13), and the GUI fetches only the changed lines (steps 14–15). This is why our notation has a separate publish–subscribe arrow. Use Case 2 showed the proxy panes and the codec protocol handshake that make up the client–server side of the system.

Step 3: Building the box-and-arrow diagram. We merged the two sequence diagrams into one static view (Figure 1). Every lifeline became a box and every kind of message between two components became a dependency arrow, marked as a procedural call, a configuration injection or a publish–subscribe notification. We then checked the diagram in the other direction: every arrow had to be used by at least one sequence diagram or be supported by the documentation. Finally, we grouped the boxes into four tiers to make the diagram easier to read. These tiers are a visual grouping, not a layered style (Section 6.3).

3 Conceptual Architecture

Figure 1 shows WezTerm's conceptual architecture: eight subsystems arranged in four tiers, with a fifth band at the bottom for the external systems WezTerm depends on but does not contain.

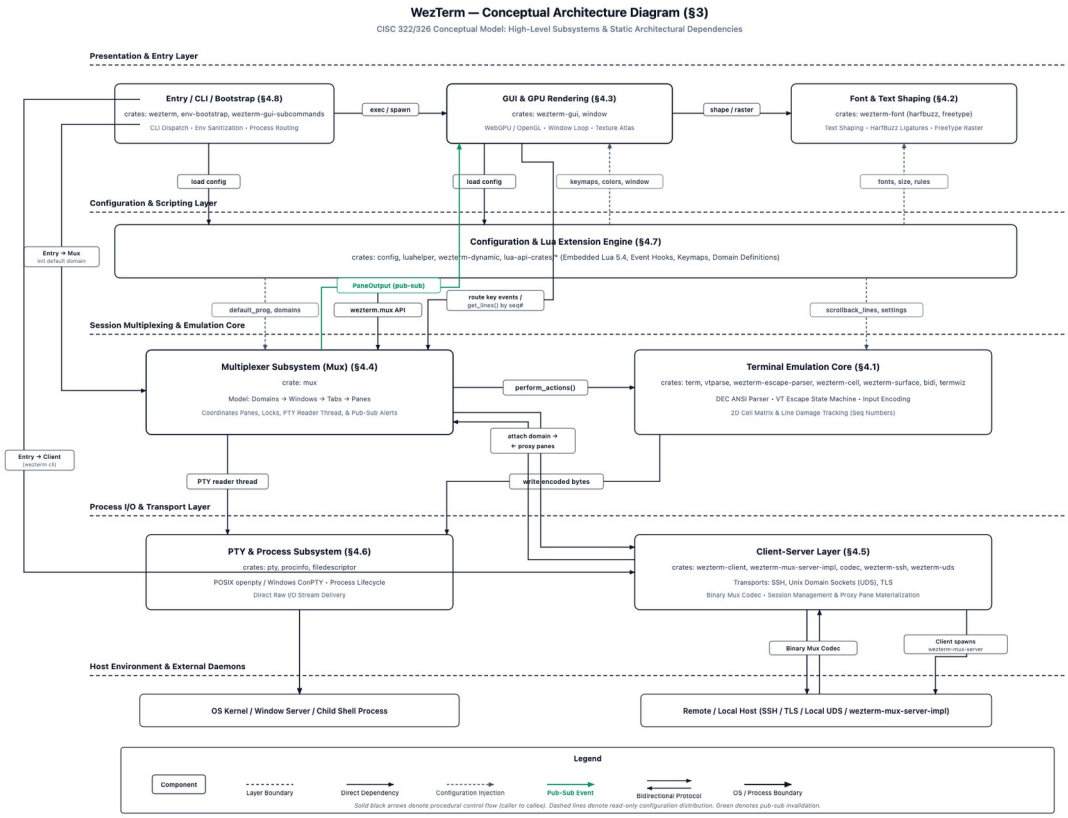


Figure 1: Conceptual architecture of WezTerm (box-and-arrow diagram)

3.1 Architectural Tiers

- **Presentation & Entry:** Entry/CLI/Bootstrap, GUI & GPU Rendering and Font & Text Shaping. This tier handles process launch, native windows and input, and drawing text.
- **Configuration & Scripting:** the Configuration & Lua Extension Engine, a cross-cutting source of settings and scripting for every other tier.
- **Session Multiplexing & Emulation Core:** the Multiplexer and the Terminal Emulation Core, which hold the session model (domains, windows, tabs and panes) and the screen model (cells, lines and scrollback).
- **Process I/O & Transport:** PTY & Process and the Client–Server Layer, which isolate local process I/O and network or IPC transports from the core.
- **Host Environment & External Daemons:** the OS kernel, window server and child shell, and the local or remote host running wezterm-mux-server.

3.2 Dependencies

Each arrow in Figure 1 is one dependency. Solid arrows are procedural calls, dashed arrows are read-only configuration injection, and the green arrow is a publish–subscribe notification. Step numbers refer to the sequence diagrams in Section 5.

- **Entry → Configuration, GUI, Mux and Client–Server.** At startup Entry evaluates wezterm.lua, hands window-related subcommands to wezterm-gui (by exec or spawn), builds the Mux with its default LocalDomain, and lets lightweight wezterm cli commands talk to a running server through wezterm-client [5], [6].

- **GUI → Configuration.** wezterm-gui loads, and later reloads, wezterm.lua itself (UC2 step 4).
- **Configuration ⇢ GUI, Font, Mux and Terminal Core (dashed).** Configuration injects key maps, colours and window settings into the GUI; font families, sizes and rules into Font; the default program and domain list into the Mux; and scrollback and terminal settings into the Terminal Core [7].
- **Configuration → Mux (wezterm.mux API).** Lua scripts can spawn windows, split panes and switch workspaces [8].
- **GUI → Font (shape/raster).** The GUI sends runs of styled cells to be shaped and rasterized (UC1 steps 16–17).
- **GUI → Mux (route key events, get_lines() by sequence number).** Input goes to the active pane, and the GUI pulls only lines that changed (UC1 steps 2 and 14–15).
- **Mux → GUI (PaneOutput, publish–subscribe).** The Mux announces new output without calling the renderer (UC1 step 13, UC2 step 20).
- **Mux → Terminal Core (perform_actions()).** Parsed actions update the cell grid and line sequence numbers (UC1 steps 9–10).
- **Terminal Core → PTY (write encoded bytes)** and **Mux → PTY (PTY reader thread).** Encoded keystrokes are written to the PTY master (UC1 step 4), and a background thread per pane reads the shell's output as it arrives (UC1 step 7).
- **Mux ↔ Client–Server (attach domain, proxy panes).** The Mux asks wezterm-client to connect, and the client builds and updates proxy panes in the Mux (UC2 steps 8, 14 and 19).
- **External boundaries.** The Client–Server Layer talks to the host over the binary mux codec and can spawn wezterm-mux-server; PTY & Process drives the OS kernel and the child shell.

3.3 Relation to the Reference Architecture

No reference architecture for terminal emulators is posted on onQ, so we compared our architecture with the closest one in the course material, the reference architecture for web browsers [9]. Its subsystems map closely onto ours. The browser's **User Interface** corresponds to GUI & GPU Rendering; the **Browser Engine**, which coordinates the interface and the rendering engine, corresponds to the Mux; the **Rendering Engine** and its **XML Parser** correspond to the Terminal Emulation Core and its escape-sequence parser; the **Display Backend** corresponds to Font & Text Shaping together with the GPU drawing in the GUI; **Networking** corresponds to the Client–Server Layer; the **JavaScript Interpreter** corresponds to the embedded Lua interpreter; and **Data Persistence** corresponds to the configuration file.

The comparison also shows where WezTerm differs. A terminal's content comes from a local child process rather than from the network, so WezTerm needs a PTY & Process subsystem with no browser counterpart, and it adds Entry/CLI/Bootstrap to start several executables. Its Client–Server Layer synchronizes live session state with a server that outlives the GUI, whereas a browser's networking subsystem only fetches documents. The mapping also served as a completeness check on our subsystem list.

4 Subsystems and Responsibilities

4.1 Terminal Emulation Core

The Terminal Emulation Core (the term crate, with vtparse, wezterm-escape-parser, wezterm-cell, wezterm-surface and bidi) decides what should be on the screen [10]. It keeps the grid of cells, the cursor, the scrollback and the alternate screen used by full-screen programs such as vim, but it draws nothing and does not own the PTY. Parsing has two stages: vtparse, a state machine based on Paul Williams' DEC ANSI parser [11], classifies bytes as text, control codes or parts of escape

sequences, and wezterm-escape-parser turns them into typed actions such as “set foreground colour”. TerminalState then applies each action: it writes characters, wraps lines, applies colours, scrolls rows into scrollback, tracks inline images and hyperlinks, and rewraps text on resize [12].

Every line carries a sequence number that increases when the line changes. This single mechanism lets the GUI redraw only dirty lines and lets the mux server send only changed lines to remote clients. The core also works in reverse, encoding key and mouse events according to modes the running program has set, such as application cursor keys and bracketed paste. Because it has no GUI dependency, the same model runs inside wezterm-mux-server and keeps a session up to date while no client is attached, which is what makes reconnection possible.

4.2 Font & Text Shaping

Font & Text Shaping (the wezterm-font crate) decides how characters look. It finds a font for each character, shapes text into positioned glyphs with HarfBuzz, rasterizes glyphs into bitmaps with FreeType, and measures the main font to set the cell size, which decides how many rows and columns fit in the window [13], [14]. Users choose fonts in Lua with wezterm.font or wezterm.font_with_fallback, and font_rules can select other faces for bold or italic text. WezTerm searches the directories in font_dirs as well as the system font locator; setting font_locator to “ConfigDirsOnly” makes a configuration behave the same on every machine [15]. Because no single font covers all of Unicode, fonts are listed in order as fallbacks.

Shaping and rasterizing are expensive, so their results are cached in a GPU texture atlas; ordinary typing rarely triggers new work, while a font-size or DPI change invalidates the cache. The mux server never uses this subsystem. In a remote session all shaping happens on the client with the client’s own fonts, so the same session can look different on two machines.

4.3 GUI & GPU Rendering

GUI & GPU Rendering (the window and wezterm-gui crates) owns the native windows, captures input and draws the terminal. The window crate wraps each platform’s windowing system (Win32, Cocoa, Wayland and X11), handles resizing, DPI changes and the clipboard, and turns raw platform events into WezTerm input events [10]. wezterm-gui evaluates key bindings, lays out tab bars, split borders and overlays such as the launcher and copy mode, and routes all other input to the active pane in the Mux.

Rather than polling, the GUI subscribes to output notifications from the Mux. When a pane reports new output, it fetches only the lines whose sequence numbers changed, groups their cells into runs with matching attributes, and sends those runs to Font & Text Shaping. It packs the returned glyph bitmaps into its texture atlas, batches cells into instanced quads and issues draw calls. The renderer is chosen with the front_end setting: OpenGL is the default, WebGPU is an alternative, and a software renderer remains as a fallback [16]. Window management must run on the platform’s main thread, so PTY reading and parsing happen on background threads and many rapid updates are combined into one frame per display refresh.

4.4 Multiplexer (Mux)

The Multiplexer (the mux crate) is WezTerm’s central model of every running session. It is a single process-wide object that arranges sessions into domains, windows, tabs and panes. A domain describes where sessions live: a LocalDomain runs programs on this machine, while other domains connect to SSH hosts or to a mux server. Windows hold tabs and can be grouped into workspaces, and each tab holds a binary split tree of panes [2].

The key design choice is that every pane implements a common Pane trait. A local pane wraps a Terminal and a PTY; a remote pane is a proxy for a pane on a server. The GUI treats both the same way, so multiplexing did not require changes to the renderer. The Mux also manages concurrency: for each local pane it starts a reader thread that reads the PTY, parses the output into actions and applies them to the pane’s Terminal, and then broadcasts a notification that the pane

has new output. When a pane's process exits, the Mux removes the pane and cleans up empty tabs and windows. The Mux reads its default program and domain list from Configuration, and Lua scripts can drive it through the wezterm.mux API [8].

4.5 Client–Server Layer

The Client–Server Layer separates a session's state from the window that shows it. A headless wezterm-mux-server runs its own Mux with real Terminals and PTYs, and a GUI attaches through a client domain provided by wezterm-client, seeing proxy panes that mirror the server's panes. Because the server owns the state, sessions survive the GUI closing, a dropped connection or a laptop going to sleep, and wezterm connect restores the same windows, tabs, scrollback and running programs [2].

Client and server exchange typed messages called PDUs, defined in the codec crate. A connection starts with a version handshake; the client then lists the server's panes and layout and builds matching proxy panes. To keep bandwidth low, the server never streams raw PTY output or pixels: it reports which lines changed, the client requests only the lines it needs, and shaping always happens on the client. The protocol is independent of its transport: Unix domain sockets (wezterm-uds) for a local server, SSH (wezterm-ssh) for a remote host, and TLS [2]. The wezterm cli commands use the same protocol.

4.6 PTY & Process

PTY & Process (the pty, procinfo and filedescriptor crates) creates pseudo-terminals and launches child programs such as shells. On Unix-like systems a terminal emulator opens a PTY pair and starts the child on the slave side; on Windows, ConPTY provides the same function [4]. The subsystem carries input from WezTerm to the child and output back, leaving interpretation to the Terminal Emulation Core and session management to the Mux.

In Use Case 1 the Terminal Core writes the encoded keystroke to the PTY master and the kernel delivers it to the shell's standard input (steps 4–5); the shell's output returns through the PTY (step 6) and is read by the Mux's reader thread (step 7). Because a child process can produce output at any time, PTY I/O cannot follow one step-by-step GUI flow, which is why Figure 1 shows asynchronous reading between the Mux and this subsystem. In Use Case 2 the subsystem is involved only indirectly: on the server, the PTYs of existing panes keep running while no client is attached, and their output becomes visible again once a client reconnects.

4.7 Configuration & Lua Engine

Configuration & Lua (the config, wezterm-dynamic and lua-api-crates crates) [17] turns the user's wezterm.lua into the settings every other subsystem uses. WezTerm has no settings menu: its configuration is a Lua program that returns a table of options. The subsystem locates the file (from --config-file, the WEZTERM_CONFIG_FILE variable or a standard location), runs it in an embedded Lua interpreter, and converts the result into a validated configuration with defaults for missing options; config_builder can warn about or reject misspelled options [7], [18].

Nearly every component depends on it: the GUI reads key bindings, colours and window settings; Font reads font choices; the Terminal Core reads the scrollback size; the Mux and PTY use the default program, working directory and environment; and the Client–Server Layer reads domain definitions. The dependency also runs the other way, since scripts can create windows and panes through wezterm.mux and register callbacks with wezterm.on, for example to format tab titles [8], [19]. WezTerm watches the configuration file and reloads it when it changes, rebuilding the Lua state and clearing old event handlers. If the file fails to load, WezTerm shows an error and falls back to the defaults instead of crashing [7], [18].

4.8 Entry / CLI / Bootstrap

Entry/CLI/Bootstrap (the wezterm, env-bootstrap and wezterm-gui-subcommands crates) is where every WezTerm process starts, whether it is the CLI, the GUI or the mux server. All three executables first run the same bootstrap routine, which sets up logging and a crash handler, records where the executables live, cleans the environment that child processes will inherit (for example removing stale WINDOWID and VTE_VERSION variables), and registers the Lua API modules [5].

WezTerm ships as wezterm, for use from a terminal, and wezterm-gui, for launching from a desktop, because on Windows a graphical program cannot print to the console that started it [6]. The wezterm binary parses the command line: lightweight commands such as wezterm cli run in the CLI process, while commands that need a window, such as start, connect and ls-fonts, are handed to wezterm-gui with the same arguments (by exec on Unix and by spawning on Windows). Once wezterm-gui runs, it loads the configuration, builds the Mux with its default domain, and checks whether a matching GUI is already running; if so, it asks that instance to open the new tab over the client–server protocol and exits [5]. After startup, Entry takes no part in normal operation. Its centralized dispatch follows a front-controller pattern, so a new subcommand only has to be added in one place.

5 Use Cases

The two use cases exercise different parts of the architecture. Use Case 1 follows the interactive loop inside a single process; Use Case 2 crosses a process boundary to reattach to a running server. In both sequence diagrams, solid arrows are requests, dashed arrows are replies and green arrows are publish–subscribe events.

5.1 Use Case 1: Typing a Command in a Local Tab

Precondition: WezTerm is already running, and the configuration and local Mux domain were initialized at startup (Section 4.8).

Input path (steps 1–6). The user presses a key and the window crate delivers the event to wezterm-gui (step 1). The GUI checks its key bindings and, since the key is not a GUI shortcut such as opening a tab, routes the event to the active pane in the Mux (step 2). The Mux asks the pane's Terminal to encode the key according to the current terminal modes (step 3), and the Terminal Core writes the bytes to the PTY master (step 4). The kernel delivers them to the shell's standard input (step 5), and the shell runs the command and writes text and escape sequences back to the PTY (step 6).

Output path (steps 7–13). A reader thread owned by the Mux wakes when the PTY has data and reads the bytes (step 7). On the same thread the Mux parses them into terminal actions with vtparse and wezterm-escape-parser (step 8) and passes the actions to the Terminal with perform_actions() (step 9). The Terminal Core applies them to the cell grid, increments the sequence number of every changed line (step 10) and returns (step 11). The Mux raises a PaneOutput event (step 12) and notifies its subscribers, including the GUI (step 13). Parsing on a background thread keeps heavy output from freezing the window.

Rendering (steps 14–19). The GUI calls get_lines() (step 14) and receives only the visible lines whose sequence numbers changed (step 15). It sends runs of cells with matching attributes to Font & Text Shaping, where HarfBuzz shapes them and FreeType rasterizes any glyph not yet cached (steps 16–17). The GUI uploads new glyphs to its texture atlas and dispatches its shaders (step 18), and the frame is presented to the user on the next V-Sync (step 19).

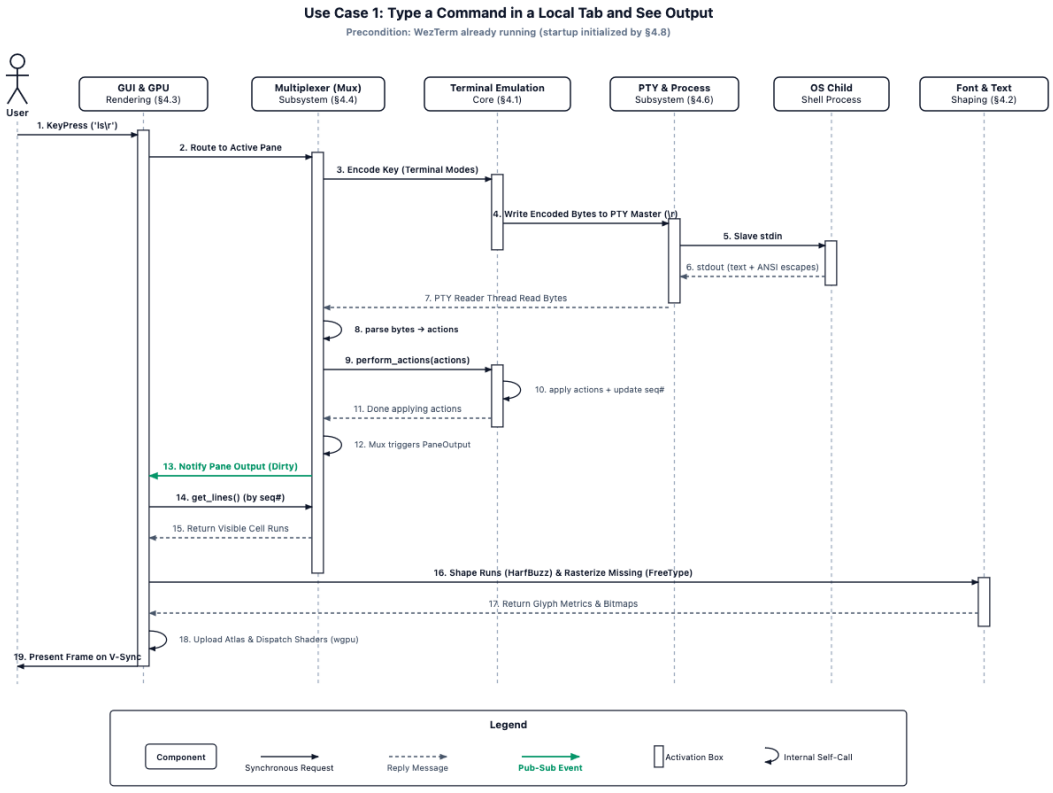


Figure 2: Use Case 1 sequence diagram: typing a command in a local tab and seeing the output

5.2 Use Case 2: Reconnecting to a Saved Multiplexer Session

The diagram follows a connection over a local Unix domain socket; SSH and TLS domains use the same messages over a different transport [2].

Startup (steps 1–7). The user runs wezterm connect unix (step 1). Entry runs the bootstrap and parses the arguments (step 2), then hands the command to wezterm-gui (step 3). The GUI evaluates wezterm.lua (step 4) and receives the unix_domains definition (step 5). It turns the request into a StartCommand that attaches to that domain instead of spawning a new shell (step 6) and initializes the client domain in the local Mux (step 7).

Connection and inventory (steps 8–14). The local Mux asks wezterm-client to establish a connection (step 8). The client sends GetCodecVersion over the socket and the server replies, so both sides agree on the protocol before any session data is exchanged (steps 9–10). The client then sends ListPanes (step 11); the server-side Mux looks up its window and tab layout (step 12) and returns the windows, tabs and panes it owns (step 13). The client materializes matching proxy panes in the local Mux (step 14). These implement the same Pane trait as local panes, so the GUI does not need to know they are remote.

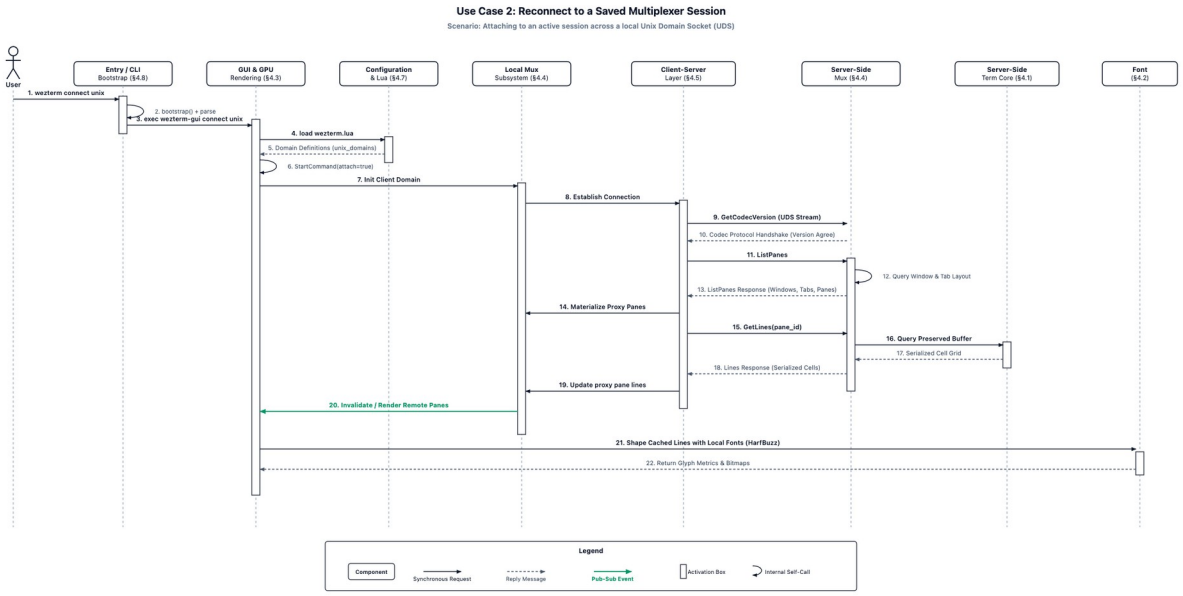


Figure 3: Use Case 2 sequence diagram: reconnecting to a saved multiplexer session over a local Unix domain socket

Content and rendering (steps 15–22). The client requests each pane's visible lines with GetLines (step 15). The server-side Mux reads the preserved buffer from that pane's Terminal Core (steps 16–17) and returns the serialized cells (step 18); the server sends parsed lines, never raw escape bytes or pixels. The client writes the lines into the proxy panes (step 19), and the local Mux sends an invalidate notification to the GUI (step 20), the same publish–subscribe mechanism as in Use Case 1. The GUI shapes the lines with the client's own fonts (steps 21–22) and draws the restored session as in Use Case 1. From then on, when programs on the server produce output, the server reports which lines changed and the client pulls them with GetLines.

6 Architectural Styles

WezTerm does not follow a single pure style. Our diagrams point to two main styles: client–server decides where sessions live, and implicit invocation decides how components react to change.

6.1 Client–Server

In the client–server style, a stand-alone server provides a service that clients use over a connector [3]. In WezTerm the service is session management: wezterm-mux-server owns the real terminals and PTYs, and the GUI connects as a client through a unix, SSH or TLS domain, all using one protocol [2]. Use Case 2 is this style end to end.

Benefits. Sessions can outlive the GUI, and TLS domains reconnect automatically after a dropped connection [2]. Because remote panes look local, adding a transport does not change the renderer, and the server needs no fonts or display since shaping stays on the client (UC2 steps 21–22). **Challenges.** Performance depends on the network, since each keystroke to a remote pane makes a round trip. WezTerm can predict and echo keystrokes locally once latency passes local_echo_threshold_ms, which adds client complexity and can briefly show text the server never echoed. Client and server must also stay compatible: an SSH domain needs a compatible WezTerm on the remote host [2].

6.2 Implicit Invocation (Publish–Subscribe)

In implicit invocation, a component announces an event and every procedure registered for it is called, without the announcer knowing who responds [3]. WezTerm uses this at two levels. In Use Case 1 the Mux announces that a pane changed (steps 12–13), and the subscribed GUI fetches only the changed lines (steps 14–15); the Mux never calls the renderer directly. At the user level, wezterm.on lets Lua scripts register callbacks for events such as tab-title formatting [19].

Benefits. The Mux does not depend on the GUI, which is what makes a headless mux server possible. Shells can produce output at any time, and the GUI redraws only when notified. Users can extend WezTerm by registering a handler, without changing Rust code. **Challenges.** Control flow is harder to trace: in Use Case 1 the keystroke-to-frame path passes through a thread and a notification, which is why our diagrams need a separate publish–subscribe arrow. Handlers run in registration order, but one that returns false skips later handlers and the default action, so one script can silently override another, and reloading the configuration clears every handler [19].

6.3 Why Not Layered

Figure 1 groups subsystems into tiers, which suggests a layered style. We treat the tiers as a grouping rather than a style because dependencies skip tiers: Entry calls the Mux and the Client–Server Layer directly, the Mux notifies the GUI upward, and Configuration is used by every tier. A strict layered style would only let each layer use the one directly below it [3].

7 Alternative Architectural Decisions

7.1 GUI-Owned Sessions Instead of Client–Server

The GUI process could own all sessions, PTYs and shells directly, with no separate mux server. This would be simpler: fewer processes, and no protocol, transport, version compatibility or proxy panes to maintain. However, sessions would end when the GUI closed, and remote multiplexing and reconnecting after a disconnection would be much harder to support [2]. For WezTerm, persistent and reconnectable sessions justify the extra complexity, so this alternative was discarded.

7.2 Direct Calls or Polling Instead of Publish–Subscribe

The Mux could call the GUI directly whenever output arrives, or the GUI could poll every pane on a timer. Direct calls make control flow easy to follow, and polling is simple to reason about. However, direct calls would couple the Mux to the renderer and make a headless mux server impossible without special cases, while polling wastes work checking panes that have not changed and delays redraws until the next check. Publish–subscribe avoids both problems, which suits WezTerm’s asynchronous output and the separation between the Mux and the GUI.

8 Conclusion

WezTerm’s architecture is organized around the Multiplexer. A common Pane interface lets the GUI treat local and remote sessions the same way, the client–server split lets sessions outlive the window that shows them, and publish–subscribe notifications let the Mux report new output without depending on the renderer. Around this core, the Terminal Emulation Core and Font & Text Shaping separate what is on screen from how it looks, Configuration & Lua makes almost every behaviour scriptable, and Entry/CLI/Bootstrap starts the right subsystems for each executable.

The design trades simplicity for capability. Sessions survive disconnection, remote panes look local and heavy output does not freeze the window, but control flow is harder to trace, remote typing depends on network latency, and scripts can interact in surprising ways.

9 Lessons Learned

Trace behaviour before describing structure. An early version of our §3 write-up described the GUI as polling the Mux, because the documentation describes get_lines() but not what triggers it. The Use Case 1 sequence diagram shows the notification that triggers the pull, and comparing the two during the final merge exposed the contradiction. Writing the static description after tracing the behaviour would have avoided the mistake.

Small diagram changes ripple through the report. When steps were added to both sequence diagrams (parsing in Use Case 1, updating proxy panes in Use Case 2), step numbers cited in several sections no longer pointed to the right arrows. We learned to treat step numbers as shared references, update them in one pass once the diagrams were final, and check every section against the final figures rather than earlier drafts.

Crates are not the conceptual architecture. The crate list was a useful starting point, but some subsystems span many crates and some crates serve several subsystems. We had to define subsystems by responsibility rather than by folder.

Six writers need one vocabulary. Early drafts used different names for the same components and tiers. Agreeing on box and tier names from the final diagram and using them everywhere made the report consistent, and checking each section against WezTerm's documentation and source caught statements that had become outdated.

10 Data Dictionary

- **Terminal emulator:** A program that turns keystrokes into input for a shell and draws the shell's output as text on screen.
- **Pseudo-terminal (PTY):** A pair of operating-system devices (master and slave) that lets a program act as a terminal for a child process.
- **Escape sequence:** Bytes, usually starting with ESC, that a program sends to control colours, cursor movement and other terminal behaviour.
- **Domain, window, tab, pane:** WezTerm's session hierarchy: a source of sessions, a top-level window, a tab within it, and a single terminal session within a tab.
- **Proxy pane:** A local stand-in for a pane that lives on a mux server.
- **Sequence number:** A per-line counter that increases when the line changes, used to find lines that need redrawing.

11 Naming Conventions

- **UC1 / UC2:** Use Case 1 and Use Case 2 (Section 5).
- **Mux:** The Multiplexer subsystem (mux crate).
- **GUI / GPU:** Graphical user interface; graphics processing unit.
- **PDU:** Protocol data unit, a typed message in WezTerm's codec protocol.
- **UDS / SSH / TLS:** Unix domain socket, Secure Shell and Transport Layer Security, the transports for remote domains.
- **wezterm-*:** Prefix for WezTerm crates and executables, such as wezterm-gui and wezterm-mux-server.

References

[1] W. Furlong et al., “WezTerm source code repository,” GitHub. https://github.com/wezterm/wezterm

[2] WezTerm, “Multiplexing,” WezTerm Documentation. https://wezterm.org/multiplexing.html

[3] CISC 322/326 lecture notes, “Architecture Styles,” onQ, Queen’s University, Fall 2026.

[4] WezTerm, “What is a Terminal?,” WezTerm Documentation. https://wezterm.org/what-is-a-terminal.html

[5] WezTerm, source files wezterm/src/main.rs, env-bootstrap/src/lib.rs, wezterm-gui-subcommands/src/lib.rs and wezterm-gui/src/main.rs, GitHub. https://github.com/wezterm/wezterm

[6] WezTerm, “General CLI options” and “wezterm cli,” WezTerm Documentation. https://wezterm.org/cli/general.html

[7] WezTerm, “Configuration files,” WezTerm Documentation. https://wezterm.org/config/files.html

[8] WezTerm, “wezterm.mux” and “mux-startup,” WezTerm Documentation. https://wezterm.org/config/lua/wezterm.mux/index.html

[9] A. Grosskurth and M. W. Godfrey, “A reference architecture for Web browsers,” in Proc. 21st IEEE Int. Conf. Software Maintenance (ICSM), 2005. Course reading, Week 5, onQ.

[10] Docs.rs, “wezterm-term,” “termwiz,” “wezterm-gui” and “window” crate documentation. https://docs.rs

[11] P. Williams, “A parser for DEC’s ANSI-compatible video terminals,” VT100.net. https://vt100.net/emu/dec_ansi_parser

[12] WezTerm, “Escape sequences,” WezTerm Documentation. https://wezterm.org/escape-sequences.html

[13] WezTerm, “Fonts,” WezTerm Documentation. https://wezterm.org/config/fonts.html

[14] WezTerm, “Font shaping,” WezTerm Documentation. https://wezterm.org/config/font-shaping.html

[15] WezTerm, “font_shaper,” “font_rasterizer,” “font_dirs” and “font_locator,” WezTerm Documentation. https://wezterm.org/config/lua/config/

[16] WezTerm, “front_end,” WezTerm Documentation. https://wezterm.org/config/lua/config/front_end.html

[17] WezTerm, “wezterm-dynamic” and “lua-api-crates” READMEs, GitHub. https://github.com/wezterm/wezterm

[18] WezTerm, “config_builder” and “add_to_config_reload_watch_list,” WezTerm Documentation. https://wezterm.org/config/lua/wezterm/

[19] WezTerm, “wezterm.on,” WezTerm Documentation. https://wezterm.org/config/lua/wezterm/on.html

Appendix D: AI Collaboration Report

D.1 Model Selection

Five of our six members (Armaan, Arnav, Avaneesh, Ibraheem and Dagim) used Anthropic's Claude (Opus 5.5, through claude.ai); Rahul used OpenAI's ChatGPT. We compared Claude, ChatGPT and Google's Gemini against what this assignment needed most: reading a lot of material at once, including WezTerm's documentation, a Rust repository split into dozens of crates, our long shared draft, and diagram files in SVG and PlantUML. Claude fit most of the group because it handles long inputs well, reads code structure well enough to connect crates such as mux, term and codec to our subsystems, and can read images, so it could check the arrows in our diagrams directly. ChatGPT is stronger at open-ended explanation and brainstorming, which suited Rahul's sections on how a PTY connects WezTerm to a shell and on alternative designs. Gemini integrates well with Google Docs, but adding a third tool would have made our verification process harder to keep consistent.

D.2 Task Assignment

We gave the AI four kinds of tasks, and none of them involved making architecture decisions. (1) **Understanding subsystems:** each member asked it to explain their subsystem against the WezTerm documentation and repository, for example how PTY & Process fits into both use cases. (2) **Checking diagrams against write-ups:** Dagim gave Claude both diagram drafts and his §4.7 notes and asked it to check the Configuration & Lua box and its arrows. (3) **Revising sections to match the final diagrams and terminology:** Rahul used ChatGPT to update §4.6 and §7. (4) **Finding inconsistencies during the final merge:** Armaan gave Claude the full draft and asked it to find places where sections contradicted each other or the final diagrams.

D.3 Interaction Protocol and Prompting Strategy

Each member used the AI for their own sections under the same rules. We always gave it our own material as context (the subsystem notes, the current diagram or the shared draft) and asked for one specific check or explanation rather than general questions about WezTerm. For example, Dagim asked it to check the Configuration & Lua box and arrows in two diagram drafts against his notes, not "what does WezTerm's config system do?" We then asked follow-up questions about why it reached a conclusion and checked its answer against the source. Every session was logged in our group Discord with one template: name, date, tool, what was asked, what it was used for, whether it was right, and what was kept or changed.

D.4 Validation and Quality Control

Nothing the AI produced was used without checking it against a primary source, usually wezterm.org or the GitHub repository; this mattered because WezTerm changes quickly and a model's training data can lag behind. Dagim's diagram review flagged a reversed wezterm.mux arrow (it should be Config → Mux) and a missing Config → Terminal Core arrow; he confirmed both against the documentation, inspected the arrowheads himself rather than trusting the AI's reading of the image, and sent the corrections to Arnav. Rahul removed claims that were more specific than our sources or diagrams supported. Armaan's merge review found that §3 described the GUI as polling the Mux while the use cases described publish–subscribe, that the tier count differed between sections, and that step numbers no longer matched the revised diagrams; each was confirmed by rereading the sections before being corrected.

D.5 Quantitative Contribution

We estimate the AI teammate's contribution by project phase. During the initial research it carried a large share of the orientation work, explaining each subsystem against the documentation and repository, which needed a lot of context at once; we put its share of that phase at about 25%. During drafting its share was small, since every section was researched and written by its owner and any wording it suggested was rewritten, so we estimate about 5%. During the final review it did more of the cross-checking, finding contradictions between sections and step numbers that no longer matched the diagrams, roughly 30% of that work, although every finding was confirmed by a person. Weighting these by the time each phase took (about 30% research, 50% drafting and 20% review) gives $0.30 \times 25\% + 0.50 \times 5\% + 0.20 \times 30\% \approx 16\%$. We therefore estimate the AI's overall contribution at about 1/7 of the project (~15%) or slightly higher. It made none of the architecture decisions.

D.6 Reflection on Human–AI Collaboration

The AI was more valuable as a reviewer than as a source of content. It sped up the two stages that need the most context at once, understanding the codebase at the start and cross-checking a report written by six people at the end, and the errors it caught were ones we had missed. Its main risk was confidence: it could produce fluent, specific claims that went beyond our sources, and it could work from outdated context after our decisions changed. Our rules of giving it our own material, verifying against wezterm.org and logging every session kept that risk manageable, and the collaboration worked because the person who knew each subsystem stayed in charge of it.